✨ CSS Transitions

Smooth animations made simple with interactive examples and live demos

What are CSS Transitions?

CSS Transitions provide a way to smoothly animate changes to CSS properties over a specified duration. Instead of instantly changing from one value to another, transitions create a smooth interpolation between states, enhancing user experience and providing visual feedback.

Transitions are ideal for simple state changes (like hover effects, focus states, and active states) and require no JavaScript to implement.

Browser Support All modern browsers support CSS Transitions. Use `-webkit-` prefix for older Safari and mobile browsers for better compatibility.

Core Transition Properties

1. transition-property

Specifies which CSS properties should be transitioned.

```
Syntax: transition-property: property1, property2, ...;

Values:
• all – Transitions all animatable properties
• specific property – background-color, transform, opacity, etc.
• none – No transitions
```

Examples:

- `transition-property: background-color;`
- `transition-property: width, height, opacity;`
- `transition-property: all;`

2. transition-duration

Defines how long the transition takes to complete.

```
Syntax: transition-duration: time;

Units: seconds (s) or milliseconds (ms)
Default: 0s (instant)
```

Examples:

- `transition-duration: 0.5s;`
- `transition-duration: 300ms;`
- `transition-duration: 1s, 0.5s;` (multiple durations)

3. transition-timing-function

Controls the speed curve of the transition animation.

```
Built-in Values:
• linear – Constant speed throughout
• ease – Slow start/end, fast middle (default)
• ease-in – Slow start, fast end
• ease-out – Fast start, slow end
• ease-in-out – Slow start and end, fast middle
• step-start, step-end – Discrete steps
• steps(n, direction) – Custom step animation
```

Custom Timing:

- `transition-timing-function: cubic-bezier(0.25, 0.1, 0.25, 1);`
- `transition-timing-function: steps(4, end);`

4. transition-delay

Specifies a delay before the transition starts.

```
Syntax: transition-delay: time;

Units: seconds (s) or milliseconds (ms)
Default: 0s (starts immediately)
```

Examples:

- `transition-delay: 0.2s;`
- `transition-delay: 100ms;`
- `transition-delay: 0s, 0.1s, 0.2s;` (staggered delays)

5. transition (Shorthand)

Combines all transition properties in one declaration.

```
Syntax: transition: [property] [duration] [timing-function] [delay];

Example:
transition: background-color 0.5s ease 0.1s;
transition: all 0.3s ease-in-out;
```

🎬 Interactive Transition Demos

Color Transition
hover over me

Size Transition
hover over me

Multi-Property
hover over me

Timing Function Comparison

Hover over each box to see different timing functions (all 1 second duration)

Linear
Constant speed

Ease
Default: smooth

Ease-In
Slow start

Ease-Out
Fast start

Ease-In-Out
Smooth both ends

Staggered Transitions with Delays

Hover over the container to see staggered animation

Button Hover Effect

Hover Me!

Common Use Cases & Patterns

Hover Button Effect

```
HTML:
<button class="btn">Click Me</button>

CSS:
.btn {
  background: #0084ff;
  color: white;
  border: none;
  padding: 10px 20px;
  cursor: pointer;
  transition: all 0.3s ease;
}
.btn:hover {
  background: #0066ff;
  transform: scale(1.05);
  box-shadow: 0 5px 15px rgba(0, 212, 255, 0.4);
}
```

Form Input Focus Effect

```
CSS:
input {
  border: 2px solid #ccc;
  padding: 10px;
  transition: border-color 0.3s ease, box-shadow 0.3s ease;
}
input:focus {
  outline: none;
  border-color: #0084ff;
  box-shadow: 0 0 10px rgba(0, 212, 255, 0.3);
}
```

Card Hover Lift Effect

```
CSS:
.card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}
.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 10px 25px rgba(0,0,0,0.15);
}
```

Smooth Color Change

```
CSS:
.element {
  background: red;
  transition: background-color 0.5s ease;
}
.element.active {
  background: green;
}
```

Menu Item Underline Animation

```
CSS:
a {
  text-decoration: none;
  border-bottom: 2px solid transparent;
  transition: border-bottom-color 0.2s ease;
}
a:hover {
  border-bottom-color: #0084ff;
}
```

Best Practices & Tips

- **Keep Duration Short:** 0.2s - 0.5s for most UI interactions. Users notice delays longer than 1s.
- **Use Ease for Natural Motion:** `ease` and `ease-in-out` feel more natural than `linear`.
- **Transition All vs Specific:** Use specific properties instead of `all` for better performance (avoid transitioning properties like width/height if not needed).
- **Performance:** Animate `transform`, `opacity`, and `color`. Avoid animating `width`, `height`, `left`, `top`.
- **Delay Appropriately:** Use delays to create sequences, but keep them short (100-300ms per item).
- **Accessibility:** Respect `prefers-reduced-motion` for users who prefer minimal animation.
- **Combine with Transforms:** Use `transform: scale()` instead of `width/height` for better performance.
- **Test Across Devices:** Transitions may feel sluggish on older devices. Test on mobile.

Understanding Timing Functions

Timing functions determine how the intermediate values of CSS properties are calculated during a transition.

Function	Description	Use Case
`linear`	Same speed from start to end	Continuous, mechanical animations
`ease`	Slow start, fast middle, slow end	Most UI interactions (default)
`ease-in`	Slow start, accelerates	Elements moving away from view
`ease-out`	Fast start, decelerates	Elements entering the view
`ease-in-out`	Slow at both ends	Smooth, refined interactions
`steps()`	Discrete steps instead of smooth	Sprite animations, frame-by-frame
`cubic-bezier()`	Custom curve with control points	Fine-tuned, specific animations

Advanced Techniques

Cubic Bezier Curves

Create custom timing functions using cubic-bezier with four parameters (x1, y1, x2, y2)

```
/* Bouncy effect */
transition: all 0.6s cubic-bezier(0.68, -0.55, 0.265, 1.55);

/* Elastic effect */
transition: all 0.5s cubic-bezier(0.175, 0.885, 0.32, 1.275);

/* Precise control */
transition: all 0.3s cubic-bezier(0.25, 0.1, 0.25, 1);
```

Steps Function

Create frame-by-frame animations:

```
/* Sprite animation with 4 steps */
transition: background-position 0.8s steps(4, end);

/* Loading animation */
transition: transform 1s steps(8, end) infinite;
```

Multiple Transitions with Different Timing

```
.element {
  transition-property: background-color, transform, box-shadow;
  transition-duration: 0.3s, 0.5s, 0.4s;
  transition-timing-function: ease, ease-out, ease-in;
  transition-delay: 0s, 0.1s, 0.2s;
}
```

Respecting User Preferences

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Browser Compatibility

Modern Browsers: All modern browsers (Chrome, Firefox, Safari, Edge) support CSS Transitions natively without prefixes.

Older Browsers: Use `-webkit-` prefix for Safari 5-8, iOS Safari, and older Android browsers.

Internet Explorer: IE 10+ supports CSS Transitions (no prefix needed in IE 10+).

Vendor Prefixes Example

```
.element {
  -webkit-transition: all 0.3s ease;
  -moz-transition: all 0.3s ease;
  -ms-transition: all 0.3s ease;
  -o-transition: all 0.3s ease;
  transition: all 0.3s ease;
}
```

Quick Reference Summary

Property	Purpose	Default
`transition-property`	Which properties to animate	all
`transition-duration`	How long animation lasts	0s
`transition-timing-function`	Speed curve of animation	ease
`transition-delay`	Delay before animation starts	0s
`transition`	Shorthand for all properties	N/A

Common Mistakes to Avoid

- **Forgetting to set duration:** Transitions default to 0s (instant). Always set `transition-duration`.
- **Using `all` unnecessarily:** Specify properties for better performance. `transition: all 0.3s;` triggers for every property change.
- **Transitioning non-animatable properties:** Properties like `display: none` can't be transitioned. Use `opacity` and `visibility` instead.
- **Ignoring timing functions:** Don't always use `linear`. `ease` feels more natural.
- **Excessive delays:** Delays longer than 0.5s feel sluggish. Keep them short.
- **Not testing on mobile:** Transitions may perform poorly on older devices. Test thoroughly.
- **Animating layout properties:** Avoid transitioning `width`, `height`, `left`, `top`. Use `transform` instead.